



เอกสารรายงานฉบับสมบูรณ์

เรื่อง

เปรียบเทียบการสร้าง Unit Tests ด้วยอัลกอริทึมและ AI บน Defects4J

โดย

นายธนินธร	อันทรบุตร	673380043-6	sec.1
นายศุภกร	กรมรินทร์	673380061-4	sec.1
นายณัฏพล	เพ็งพล	673380267-4	sec.1
นายณัฐกรณ์	อินธิสาร	673380268-2	sec.1

เสนอ

ผศ.ดร.จิตสุธา สุ่มเล็ก

รายงานนี้เป็นส่วนหนึ่งของรายวิชา CP353201 Software Quality Assurance
ภาคเรียนที่ 1 ปีการศึกษา 2569 สาขาวิชาวิทยาการคอมพิวเตอร์ วิทยาลัยการคอมพิวเตอร์
มหาวิทยาลัยขอนแก่น

บทคัดย่อ

วัตถุประสงค์ โครงการนี้ศึกษาและเปรียบเทียบการสร้างชุดทดสอบระดับหน่วยของโปรแกรมภาษา Java ด้วยอัลกอริทึม CMA-ES และ FSCS-ART กับปัญญาประดิษฐ์ Sonnet 5 และ Gemini 3.5 Flash Lite ที่เรียกผ่าน KKU IntelSphere โดยใช้ Defects4J รุ่น 3.0.1 เป็นแหล่งบั๊กจริง

วิธีการ ประเมินความถูกต้องบนรุ่นที่แก้ไขแล้ว (fixed) ความสามารถในการตรวจพบข้อบกพร่องบนรุ่นที่ยังมีบั๊ก (buggy) ความครอบคลุมของโค้ด เวลา และปริมาณ tokens ที่บริการรายงาน กำหนดเป้าหมาย 500 บั๊ก คิดเป็น 2,000 งานเมื่อใช้ทั้งสี่วิธี

ผลการทดลอง ผลที่รวบรวมเมื่อวันที่ 4 ตุลาคม 2569 เวลา 21:44 น. มี 130 บั๊กจาก 17 projects รวม 396 งานที่มีสถานะบันทึกแล้ว (19.80% ของเป้าหมาย) โดย 68 บั๊กมีสถานะครบทั้งสี่วิธี และ 9 บั๊กประเมินสำเร็จครบทั้งสี่วิธี CMA-ES และ FSCS-ART ประเมินสำเร็จวิธีละ 75 บั๊ก ตรวจพบข้อบกพร่อง 10 และ 9 บั๊กตามลำดับ ส่วน Sonnet ประเมินสำเร็จ 27 บั๊กและตรวจพบ 3 บั๊ก และ Gemini ประเมินสำเร็จ 26 บั๊กและตรวจพบ 3 บั๊ก

ข้อสังเกตสำคัญ เมื่อเปรียบเทียบเฉพาะ 9 บั๊กที่ประเมินครบ AI มีค่าเฉลี่ย line และ condition coverage สูงกว่าอัลกอริทึม แต่ทั้งสี่วิธีไม่ตรวจพบบั๊กในชุดย่อยดังกล่าว จึงยังสรุปไม่ได้ว่า coverage ที่สูงกว่าหมายถึงการตรวจพบบั๊กที่ดีกว่า

ข้อจำกัด จำนวนบั๊กที่ประเมินได้ต่างกันในแต่ละวิธี ใช้ oracle คนละแนวทาง สร้างเพียงหนึ่งรอบต่อบั๊กต่อวิธี และโควตา AI ไม่เพียงพอสำหรับงานต่อเนื่อง รายงานนำเสนอผลจริงพร้อมหลักฐานการทำซ้ำโดยไม่อ้างว่าทดลองครบ 500 หรือ 854 บั๊ก

สารบัญ

1. ชื่อและข้อมูลรายงาน.....	1
2. หลักการและเหตุผล.....	1
3. วัตถุประสงค์ของโครงการ.....	2
3.1 พัฒนา CMA-ES และ FSCS-ART ให้สร้าง JUnit test suites สำหรับเป้าหมาย Java ที่รองรับใน Defects4J.....	2
3.2 ออกแบบและใช้พรมแดนเดียวกันภายในเงื่อนไขการทดลองกับ AI สองตัวผ่าน KKU IntelSphere	2
3.3 วัดความถูกต้องบน fixed revision การตรวจพบบั๊ก coverage เวลา และ tokens จากการรันจริง.....	2
3.4 เปรียบเทียบผลทั้งภาพรวมและบั๊กชุดเดียวกัน พร้อมวิเคราะห์ปัญหาและเก็บหลักฐานให้ผู้อื่นทำซ้ำได้	2
4. ทฤษฎีและงานที่เกี่ยวข้อง.....	2
4.1 Unit testing และ test oracle	2
4.2 CMA-ES	2
4.3 การแปลงเวกเตอร์เป็นเทส.....	3
4.4 FSCS-ART	3
4.5 Generative AI และพรมแดน.....	3
4.6 Defects4J และงานที่เกี่ยวข้อง	3
5. วิธีดำเนินการทดลอง.....	4
5.1 การกำหนดขอบเขตและหน่วยนับ	4
5.2 เครื่องมือและสภาพแวดล้อม	4
5.3 การเตรียม target และการตรวจ suite.....	5
5.4 ขั้นตอนและข้อกำหนดพรมแดน AI.....	6
5.5 เงื่อนไข AI และการตั้งค่า.....	6
5.6 ตัวชี้วัดและตัวหาร	7
5.7 การจำแนกสถานะ	8
5.8 ฐานันท์ของผลสำเร็จและผลบางส่วน.....	8
5.9 การตรึงหลักฐานและการย้าย repository	9
6. ผลการทดลอง.....	9
6.1 จำนวนที่ทำงานจริงและความครอบคลุม projects	9
6.2 ผลรวมของแต่ละวิธี.....	10

6.3 ความครอบคลุมของโค้ด	11
6.4 เวลาและทรัพยากร AI	12
7. การวิเคราะห์เปรียบเทียบ	13
7.1 เปรียบเทียบบนบั๊กชุดเดียวกัน	13
7.2 ความแตกต่างของการตรวจพบบั๊ก	14
7.3 ตัวอย่างหลักฐานที่ตรวจสอบย้อนกลับได้	14
8. ปัญหาและสิ่งที่เรียนรู้	15
8.1 ปัญหาที่พบจากผลจริง	15
8.2 ผลลัพธ์การเรียนรู้	15
9. ขอบเขต ทรัพยากร และข้อจำกัด	16
9.1 ขอบเขตที่ดำเนินการแล้ว	16
9.2 ข้อจำกัดของการเปรียบเทียบ	16
9.3 ทรัพยากรและสถานที่ดำเนินงาน	16
9.4 งานที่ค้างอยู่	17
10. สรุปผลและข้อเสนอแนะ	17
10.1 ความเชื่อมโยงกับสิ่งส่งมอบรอบที่ 2	18
11. เอกสารอ้างอิง	18
ภาคผนวก ก สถานะรายบั๊ก	20
ภาคผนวก ข การทำซ้ำและหลักฐาน	25
ข.1 เริ่มจากชุดหลักฐานที่ตริง	25
ข.2 แสดงผลที่บันทึกไว้	25
ข.3 รัน suite เดิมผ่าน Defects4J	25
ข.4 การสร้างใหม่และการตรวจ provenance	26

รายการภาพประกอบ

ภาพที่ 1 กระบวนการสร้างเทส ตรวจสอบผล และเก็บหลักฐาน

ภาพที่ 2 จำนวนงานที่มีสถานะและจำนวนงานที่ประเมินสำเร็จ

ภาพที่ 3 ค่าเฉลี่ย coverage เฉพาะกรณีที่วัดได้

ภาพที่ 4 เวลาเฉลี่ยที่วัดได้ แยกการสร้างและการประเมิน suite

ภาพที่ 5 ค่าเฉลี่ย coverage ของ 9 บั๊กที่ประเมินครบวิธี

ตารางผลรายบั๊กฉบับเต็มและค่าดิบอยู่ใน `Report/data/final_comparison.csv` ของชุดหลักฐาน

ข้อความและตารางในรายงานฉบับนี้ใช้ผลจาก snapshot เดียวกัน

1. ชื่อและข้อมูลรายงาน

ชื่อภาษาไทย

การเปรียบเทียบการสร้างชุดทดสอบระดับหน่วยด้วยอัลกอริทึมและปัญญาประดิษฐ์สำหรับ Java projects
ในชุดข้อมูล Defects4J

ชื่อภาษาอังกฤษ AI Assisted Testing and Automatic Test Case Generation: A

Benchmark and Test Coverage Evaluation

รายวิชา CP353201 Software Quality Assurance ภาคเรียนที่ 1 ปีการศึกษา 2569

สมาชิก

- นายธนินธร อันทรบุตร รหัสนักศึกษา 673380043-6 sec.1
- นายศุภกร กรมรินทร์ รหัสนักศึกษา 673380061-4 sec.1
- นายณัฏพล เพ็งพล รหัสนักศึกษา 673380267-4 sec.1
- นายณัฐกรณ์ อินธิสาร รหัสนักศึกษา 673380268-2 sec.1

อาจารย์ผู้สอน ผศ.ดร.จิตสุธา สุ่มเล็ก

2. หลักการและเหตุผล

การสร้าง unit tests จำนวนมากด้วยมือใช้เวลาและอาจพลาดชุดข้อมูลที่ทำให้โปรแกรมทำงานผิด การสร้างเทสอัตโนมัติช่วยสำรวจอินพุตและตรวจพฤติกรรมของโปรแกรมได้อย่างเป็นระบบ ขณะที่ Generative AI สามารถใช้บริบทของโค้ดและสัญญาของ API ออกแบบอินพุตและ assertions ได้ อย่างไรก็ตาม source ที่คอมไพล์ได้ยังไม่รับประกันว่าเทสถูกต้อง ครอบคลุมโค้ด หรือพบข้อบกพร่องจริง

โครงการจึงเปรียบเทียบสองแนวทางบนบั้งจริง โดยทดสอบ suite เดียวกันกับโปรแกรมรุ่น fixed และ buggy แยกข้อผิดพลาดของตัวทดสอบออกจาก fault ของโปรแกรม และรักษาผลไม่ผ่านไว้เป็นข้อมูลประกอบการเลือกเครื่องมือ ผลที่ได้ช่วยให้เห็นทั้งประโยชน์ของการสร้างเทสและต้นทุนในการทำให้เทสนั้นใช้งานได้จริง

3. วัตถุประสงค์ของโครงการ

3.1 พัฒนา CMA-ES และ FSCS-ART ให้สร้าง JUnit test suites สำหรับเป้าหมาย Java ที่รองรับใน Defects4J

3.2 ออกแบบและใช้พารามิเตอร์เดียวกันภายในเงื่อนไขการทดลองกับ AI สองตัวผ่าน KGU IntelSphere

3.3 วัดความถูกต้องบน fixed revision การตรวจพบบั๊ก coverage เวลา และ tokens จากการรันจริง

3.4 เปรียบเทียบผลทั้งภาพรวมและบั๊กชุดเดียวกัน พร้อมวิเคราะห์ปัญหาและเก็บหลักฐานให้ผู้อื่นทำซ้ำได้

4. ทฤษฎีและงานที่เกี่ยวข้อง

4.1 Unit testing และ test oracle

Unit testing ตรวจสอบพฤติกรรมหน่วยย่อยของโปรแกรมผ่านอินพุตและเงื่อนไขคาดหวัง ส่วน test oracle เป็นเกณฑ์ตัดสินว่าผลลัพธ์ถูกต้องหรือไม่ ในงานนี้อัลกอริทึมใช้ค่าพฤติกรรมอ้างอิงที่สังเกตได้อย่างคงที่จาก fixed revision ส่วน AI สร้าง assertions จากบริบทของ buggy source และสัญญาของ API แล้วรับ feedback จากการตรวจ fixed แบบจำกัดครั้ง ทั้งสองแบบมีแหล่งข้อมูลต่างกัน จึงต้องเปิดเผยความต่างนี้ในการเปรียบเทียบ

4.2 CMA-ES

CMA-ES เป็นวิธีหาค่าพารามิเตอร์ต่อเนื่องแบบสุ่ม โดยปรับค่าเฉลี่ย covariance และ step size ตามคะแนนของตัวเลือกที่สร้างขึ้น แนวคิดพื้นฐานอธิบายโดย Hansen (2016) การประยุกต์ในโครงการแปลงเวกเตอร์ตัวเลขเป็นอินพุตของ Java adapter แล้วเลือกอินพุตที่ให้พฤติกรรมหลากหลายภายใต้งบประมาณที่กำหนด

การสุ่มเขียนได้เป็น $x \sim N(m, \sigma^2 C)$ โดย m คือค่าเฉลี่ย C คือ covariance และ σ คือ step size อย่างไรก็ตาม implementation ในโครงการใช้ Cholesky inverse L^{-1} ในการอัปเดต evolution path ของ step size ซึ่งต่างจาก symmetric inverse square root ของสูตร CMA-ES มาตรฐาน จึงรายงานเป็น implementation variant ตาม source ที่เก็บในชุดหลักฐาน และไม่อ้างว่าตรงกับสูตรต้นฉบับทุกส่วน

Fitness ที่ใช้จริงเป็นความถี่ของพฤติกรรมที่สังเกตจาก fixed revision เพื่อกระตุ้นความหลากหลายของผลลัพธ์ ไม่ใช่คะแนน code coverage โดย coverage ถูกวัดภายหลังจาก suite ที่สร้างเสร็จแล้ว ข้อจำกัดคือความหลากหลายของพฤติกรรมไม่ได้รับประกันว่าจะเข้าถึงบริเวณที่มีบั๊กหรือสร้าง assertion ที่ไวต่อการเปลี่ยนแปลงนั้น

4.3 การแปลงเวกเตอร์เป็นเทส

เวกเตอร์ที่ได้จะถูกแปลงภายใต้ขอบเขตและชนิดข้อมูลที่ adapter รองรับ ต้องสร้าง receiver และ arguments ได้ก่อนจึงเรียก target method ได้ อินพุตที่สร้างไม่ได้หรือไม่เรียกเป้าหมายต้องแยกจากเทสที่รันแล้วไม่พบบัก การใช้ budget เดียวกันช่วยควบคุมจำนวนการสำรวจ แต่จำนวน JUnit methods ที่คงไว้จริงอาจต่างกันหลังตรวจความเสถียร

4.4 FSCS-ART

Adaptive Random Testing มีแนวคิดกระจายอินพุตให้ทั่วพื้นที่ค้นหามากขึ้น แทนการสุ่มแต่ละจุดโดยไม่คำนึงถึงจุดเดิม ตามงานของ Chen, Leung และ Mak (2004) ใน implementation แบบ Fixed Size Candidate Set จะสุ่ม candidate set แล้วเลือกจุดที่มีระยะห่างขั้นต่ำจากอินพุตที่เลือกแล้วมากที่สุด

เกณฑ์ที่ใช้เขียนได้เป็น $\text{argmax}(c \in C) \min(e \in E) \text{distance}(c, e)$ โดย C เป็น candidate set และ E เป็นชุดที่เลือกแล้ว ระยะห่างถูกปรับตามขอบเขตแต่ละมิติ ในงานนี้ใช้ candidate-set size 10, seed 101 และ budget 30 vectors โดยใช้ adapter และ oracle เดียวกับ CMA-ES

FSCS-ART ไม่ต้องรับ feedback จาก coverage ในการเลือกอินพุต จุดแข็งคือสำรวจความแตกต่างของอินพุตอย่างเป็นระบบ ข้อจำกัดคือระยะห่างในเวกเตอร์อาจไม่ตรงกับ ความแตกต่างของเส้นทางหรือพฤติกรรม Java โดยเฉพาะ object, strings และสถานะของ receiver

4.5 Generative AI และพรอมต์

AI ใช้โค้ดที่มีบัก signatures และคำอธิบายงานในการสร้าง JUnit source สามารถเสนอกรณีขอบเขตและ assertions ที่เกี่ยวกับสัญญาของ API ได้ แต่ยังอาจเลือก overload ผิด ใช้ dependency ที่ไม่มี หรือคาดหวังพฤติกรรมผิด การรันบน fixed revision จึงเป็นขั้นตอนจำเป็นก่อนใช้ผล buggy ตัดสินการตรวจพบ fault

4.6 Defects4J และงานที่เกี่ยวข้อง

Defects4J เป็นฐานบักจริงและโครงสร้างสำหรับการทำการทดลอง Java แบบทำซ้ำได้ โดยแยกชุด buggy และ fixed พร้อมข้อมูลการแก้ไขและ triggering tests งานต้นฉบับของ Just, Jalali และ Ernst (2014) เสนอกรอบดังกล่าว ขณะที่ inventory รุ่น 3.0.1 ที่ใช้ในโครงการมี active bugs 854 ตัวใน 17 projects

854 หมายถึงจำนวนบักใน inventory ไม่ใช่จำนวน JUnit test methods และไม่ได้หมายถึงบักที่กลุ่มทดลองแล้วทั้งหมด การใช้หลาย methods หรือเรียก AI หลาย stages กับบักเดียวยังนับเป็นบักเดียว ส่วนการตรวจ fixed ซ้ำสองครั้งเป็นการตรวจ suite เดิม ไม่ใช่ generation round ใหม่

5. วิธีดำเนินการทดลอง

5.1 การกำหนดขอบเขตและหน่วยนับ

ใช้เฉพาะ Java projects ใน Defects4J 3.0.1 ประกาศเป้าหมาย 500 active bugs และสร้างลำดับแบบ project-sorted round robin จาก inventory เดิมเพื่อให้มีตัวแทนหลาย projects ไม่มีการเลือกบักตามผลผ่านหรือผลที่ดูดี และไม่มีการสับลำดับใหม่ระหว่างรัน

หน่วยนับหลักคือคู่บักและวิธี เช่น Csv-1 กับ CMA-ES เป็นหนึ่งงาน เป้าหมาย 500 บักจึงมี 2,000 งานสำหรับวิธี การสร้างแต่ละคู่ทำหนึ่งรอบ เก็บงานที่ไม่สำเร็จและงานที่ยังไม่เริ่มแยกกัน ผลถูกตรึงหลังหยุดเริ่มงานใหม่เวลา 21:40 น. และ suite ที่รับเข้าก่อนเวลานั้นทำเสร็จ โดย snapshot สุดท้ายเกิดเวลา 21:44 น. วันที่ 4 ตุลาคม 2569

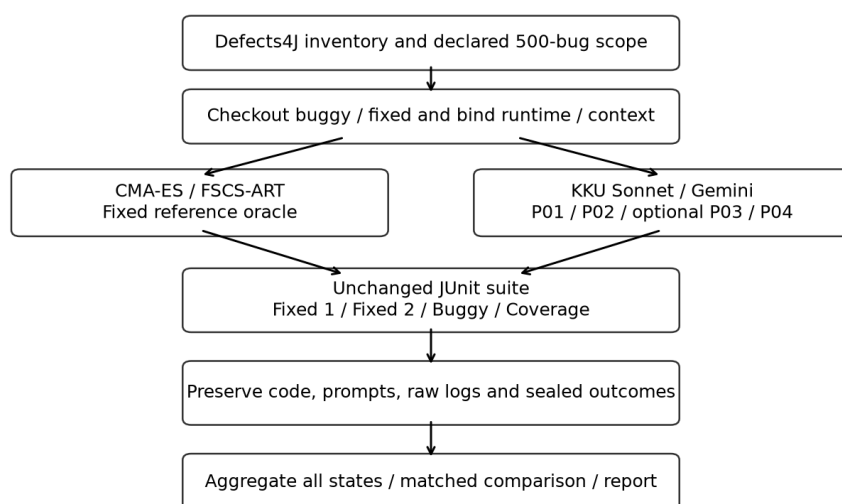
5.2 เครื่องมือและสภาพแวดล้อม

รายการ	ค่าที่ใช้
ระบบ	Windows และ Ubuntu ผ่าน WSL
CPU และ RAM	Ryzen 3 3100 4 cores / 8 threads และ RAM 16 GB
ช่องรัน Java	หนึ่ง CPU slot
Java	OpenJDK 11.0.32.1
Defects4J	3.0.1
JUnit และ source	JUnit 4 และ Java source ที่เข้ากันได้กับ Java 7
Seed และ budget	101 และ 30 vectors สำหรับอัลกอริทึม
Timeout ทดสอบ	300 วินาทีต่อ stage ตาม evaluator
Timezone ของเทส	America/Los_Angeles
Timezone ของรายงาน	Asia/Bangkok
Coverage engine	Cobertura ใน Defects4J

Timeout และ source/runtime hashes ของแต่ละคำสั่งถูกบันทึกจริงใน record.json ค่า timeout ของ API แยกจากการทดสอบ Java การใช้ CPU slot เดียวช่วยลดการแย่งทรัพยากร แต่ทำให้จำนวนงานก่อนกำหนดส่งจำกัด ไม่ได้ทำ benchmark เปรียบเทียบหลายสเปกเครื่อง

5.3 การเตรียม target และการตรวจ suite

Checkout รุ่น buggy และ fixed ของบ๊ักเดียวกัน คอมไพล์ทั้งสองรุ่น ส่งออก classpath และ modified classes แล้วค้นหา declarations ที่มีอยู่ในทั้งสองรุ่น ใช้ fixtures ที่มีอยู่จาก shared v12 (ชุด fixtures กลางรุ่นที่ 12 ที่ใช้ร่วมกันทุกวิธี) สำหรับสร้าง receiver และ arguments เฉพาะขอบเขตที่รองรับ การไม่รองรับ declaration ต้องบันทึกเป็นผลตามจริง



ภาพที่ 1 กระบวนการสร้างเทส ตรวจสอบผล และเก็บหลักฐาน

suite ที่ใช้ประเมินต้องคงเดิมระหว่าง fixed-1, fixed-2, buggy และ coverage เพื่อตรวจสอบความถูกต้องและความเสถียรเบื้องต้น ถ้า fixed ไม่ผ่านต้องหยุดการตีความ buggy ของ suite นั้น ผลที่มี NoClassDefFoundError หรือข้อผิดพลาดของ harness ไม่นับเป็น fault ของโปรแกรม

ใช้ outcome และ receipt ที่ปิดผลแล้วในการรวมตาราง การเขียน hashes ทำให้ตรวจได้ว่า source, prompts, archives และ logs ไม่เปลี่ยนภายหลัง แต่ hashes ไม่ใช่หลักฐานว่าความหมายของ assertion ถูกต้องทุกกรณี

5.4 ขั้นตอนและข้อกำหนดพรอมต์ AI

Stage	สิ่งที่ให้ AI ทำ	ขอบเขต
P01	วิเคราะห์บริบทและ target	ไม่ถือเป็น source test ที่รันแล้ว
P02	สร้าง Java และ JUnit suite	สูงสุด 12 methods ใน base
P03	แก้ compile หรือ fixed validation	ทำได้หนึ่งครั้งเมื่อจำเป็น
P04	เพิ่มกรณีทดสอบหลังวัด base	เพิ่มได้ไม่เกิน 4 methods

ข้อกำหนดสำคัญใน master prompt คือส่ง Java files ให้ครบ ใช้ assertions ที่มีความหมาย ไม่แก้ production source หรือ build ไม่เพิ่ม dependencies ไม่ใช่ ignored tests และไม่ลดความเข้มของ assertion เพียงเพื่อให้ผ่าน feedback ที่ส่งซ่อมเป็น compile/fixed validation เท่านั้น ไม่ใช่ผล buggy failure เป็นคำใบ้

ตัวอย่างสาระของคำสั่ง P02 คือ ให้สร้าง JUnit 4 ที่เข้ากันได้กับ Java 7 สำหรับ target ที่ระบุ มีกรณีปกติและขอบเขต ใช้ API ที่มีจริง และส่งไฟล์ source ที่สมบูรณ์ ข้อความที่ส่งจริงทุกคำและคำตอบที่ได้รับอยู่ในฟลเดอร์ Prompt และ Result ของแต่ละ AI จึงตรวจทำซ้ำได้โดยไม่ใช้เพียงตัวอย่างคำสั่งในรายงาน

5.5 เงื่อนไข AI และการตั้งค่า

ใช้ model IDs claude-sonnet-5 และ gemini-3.5-flash-lite ผ่าน KCU IntelSphere เท่านั้น ร้องขอ temperature 0, max_tokens 4,096 และ stream false โดย Sonnet เรียก /messages พร้อม thinking disabled ส่วน Gemini เรียก /chat/completions การรับ request สำเร็จเป็นหลักฐานว่า request ถูกยอมรับ ไม่ใช่การยืนยันค่าที่มีผลภายในบริการทุกตัว

แยก initial pilot ของ Csv-1, Lang-1 และ Math-1 ออกจาก compact nightly ใน pilot P01 มี buggy source ด้วย ส่วน nightly P01 เน้น metadata/signatures และ P02 ใช้ source เดิม พร้อมแนะนำ base ที่กระชับ 4–6 methods แต่เพดานยังเป็น 12 การวิเคราะห์รวมจึงเป็นเชิงพรรณนา และต้องเก็บ condition labels ไว้

การเข้าถึง KCU IntelSphere ใช้คีย์ที่ได้รับอนุญาตรวม 10 ชุด (หัวข้อ 6.4 อ้างถึงเป็น "คีย์ทั้งสิบ") แต่ละชุดมีโควตาและเวลาริเซตของตนเอง ซึ่งบริการไม่ได้ยืนยันโควตาหรือเวลาริเซตให้ จึงไม่ถือว่าเป็นบัญชีอิสระ 10 บัญชี

เลือกบัญชีก่อนเริ่มบักและไม่สลับบัญชีภายในงาน ใช้ข้อมูลโควตาที่บริการส่งกลับและ operator guard ก่อน stage ถัดไป ไม่มีการส่งคำขอซ้ำเมื่อผล HTTP เดิมไม่แน่นอน หากไม่มีบัญชีผ่านเกณฑ์ร่วมของสองโมเดล งาน AI ใหม่จะยังเป็น PENDING ขณะที่อัลกอริทึมทำต่อได้ คีย์ถูกเก็บในไฟล์ส่วนตัวที่ไม่ส่งขึ้น GitHub

5.6 ตัวชี้วัดและตัวหาร

ตัวชี้วัด	สูตรหรือการนับ
อัตราประเมินสำเร็จ	จำนวน DONE / จำนวน outcomes ที่บันทึกของวิธี $\times 100$
Fault detection rate	จำนวนบักที่พบ fault และมีสถานะ DONE $\div$ จำนวน DONE ของวิธี $\times 100$
Line coverage	ค่าเฉลี่ยของ $\text{line_covered} / \text{line_total} \times 100$ รายบัก เฉพาะ DONE ที่ $\text{line_total} > 0$
Condition coverage	ค่าเฉลี่ยของ $\text{branch_covered} / \text{branch_total} \times 100$ รายบัก เฉพาะ DONE ที่ $\text{branch_total} > 0$
เวลา	ค่าเฉลี่ยจากทุกงานที่มีค่าตัวเลขบันทึก แยกเวลาสร้างและเวลาประเมิน ไม่จำกัด DONE
Tokens	รวม input/output ที่บริการบันทึกจากทุกสถานะ ไม่จำกัด DONE
จำนวนบักครบวิธี	มี terminal outcome ของทั้งวิธี ไม่เท่ากับ DONE ทั้งสี่

ค่า coverage ในตารางเป็นค่าเฉลี่ยของอัตราส่วนรายบัก ไม่ใช่การนำ covered lines ทุกบักมารวมแล้วหารด้วย total lines รวม ค่าที่หายและกรณีตัวหารเป็นศูนย์ถูกตัดออกจากค่าเฉลี่ยโดยรายงานจำนวนที่ใช้คำนวณ ไม่แทนด้วยศูนย์ ค่า condition จาก Cobertura ไม่ได้แสดงว่าผ่าน MC/DC หรือครบทุกเส้นทาง

DONE หมายถึงประเมิน suite ตามขั้นตอนครบ ไม่ได้หมายถึงพบ fault ถ้า fixed ผ่านสองครั้งและ buggy ไม่พบ assertion failure จะได้ DONE กับ $\text{fault_detected}=\text{False}$ ถ้าเกิด failure จริงบน buggy จะได้ $\text{fault_detected}=\text{True}$ โดยแต่ละบักนับหนึ่งครั้ง แม้มีหลาย test methods ล้มเหลว

5.7 การจำแนกสถานะ

สถานะ	ความหมาย
DONE	ประเมินสำเร็จ มีผล coverage และ fault ที่อ่านได้
INVALID_AFTER_REPAIR	AI suite ยังไม่ถูกต้องหลังการซ่อมที่อนุญาต
INVALID_GENERATED_SUITE	suite ที่สร้างใช้ประเมินไม่ได้
UNSUPPORTED	target หรือ input/receiver อยู่นอกขอบเขตที่รองรับ
INFRA_ERROR	ผิดพลาดจาก build, classpath หรือโครงสร้างรัน
OUTPUT_INCOMPLETE	คำตอบหรือ Java source ไม่สมบูรณ์
QUOTA_PAUSED	stage ถัดไปถูกพักด้วยเกณฑ์โควตา
PENDING	ยังไม่มี outcome ที่ปิดผลใน snapshot

5.8 ฐานันท์ของผลสำเร็จและผลบางส่วน

รายงานใช้เกณฑ์ของสคริปต์รวมผลใน snapshot เดิม โดยอัตราพบ fault และค่าเฉลี่ย coverage ใช้เฉพาะงานที่มีสถานะ DONE ส่วนเวลาและ tokens ใช้ทุกงานที่มีค่าบันทึก จึงมีฐานข้อมูลที่ใช้คำนวณต่างกันในแต่ละตัวชี้วัด งานที่ไม่มีค่าจะไม่ถูกแทนด้วยศูนย์

จำนวนข้อมูลสำหรับ fault rate และ line coverage ของ CMA-ES, FSCS-ART, Sonnet และ Gemini เท่ากับ 75, 75, 27 และ 26 ตามลำดับ ส่วน condition coverage ใช้ 74, 74, 27 และ 26 งาน เวลาสร้างใช้ 88, 88, 68 และ 68 งาน และเวลาประเมินใช้ 88, 88, 55 และ 66 งาน ตามลำดับ

มีอีก 6 งานที่เก็บผล buggy ที่ใช้ดีความได้ แต่สถานะสุดท้ายไม่ใช่ DONE ได้แก่ Jsoup-4 และ Jsoup-6 ของสองอัลกอริทึม รวม 4 งาน ซึ่ง fixed ผ่านทั้งสองครั้งและ buggy ไม่พบ fault แต่ขั้นตอน coverage ล้มเหลว จึงเป็น INFRA_ERROR อีก 2 งานคือ Csv-4 และ Lang-5 ของ Sonnet ซึ่งมี base suite ที่ประเมินครบและไม่พบ fault แต่ขั้นตอนเพิ่ม suite ถูกพักด้วยโควตา จึงเป็น QUOTA_PAUSED โดย Csv-4 ใช้ repaired base

ผลบางส่วนเหล่านี้ยังคงอยู่ในหลักฐาน แต่ไม่รวมใน fault rate และ coverage ของตารางหลักตามเกณฑ์ DONE หากรวมผล buggy ที่ใช้ดีความได้ทั้ง 6 งาน ตัวหารจะเป็น 77, 77, 29 และ 26 ตามลำดับ โดยจำนวน fault ยังคงเป็น 10, 9, 3 และ 3 อย่างไรก็ตาม รายงานคงเกณฑ์ DONE เดิม ไม่เปลี่ยนสถานะย้อนหลัง และการเปรียบเทียบบักชุดเดียวกันยังใช้ 9 บักที่ DONE ครบทั้งสี่วิธี

5.9 การตรึงหลักฐานและการย้าย repository

SHA-256 ตรวจสอบ bytes ของไฟล์ รวมถึงรูปแบบการขึ้นบรรทัดใหม่ Git อาจแปลง LF/CRLF ระหว่างการจัดเก็บหรือ checkout ทำให้ hash ต่างกันแม้เนื้อหาโค้ดเหมือนเดิม ความแตกต่างดังกล่าวเพียงอย่างเดียวจึงไม่ใช่หลักฐานว่าผลทดลองเปลี่ยน ชุดแก้ไขรักษา bytes ของหลักฐานเดิมและใช้ .gitattributes ปิดการแปลงบรรทัดใหม่โดยอัตโนมัติ

6. ผลการทดลอง

6.1 จำนวนที่ทำงานจริงและความครอบคลุม projects

ผลที่ตรึงมี 130 บักจาก Java projects ทั้ง 17 รายการ รวม 396 outcomes จากเป้า 2,000 งาน คิดเป็น 19.80% ของงานที่วางแผนไว้ มีสถานะครบทั้งสี่วิธี 68 บัก และประเมินสำเร็จครบทั้งสี่วิธี 9 บัก จึงต้องแยกการมีผลทุกวิธีออกจากการมี suite ที่วัดได้ทุกวิธี

Project	บัก	ผล	CMA	FSCS	Sonnet	Gemini
Chart	8	24	4 / 0	4 / 0	4 / 0	3 / 1
Cli	8	24	0 / 0	0 / 0	0 / 0	0 / 0
Closure	8	24	0 / 0	0 / 0	0 / 0	0 / 0
Codec	8	24	7 / 2	7 / 1	0 / 0	2 / 0
Collections	8	24	8 / 1	8 / 2	4 / 1	1 / 0
Compress	8	24	6 / 2	6 / 1	2 / 0	1 / 0
Csv	8	24	4 / 0	4 / 0	2 / 0	4 / 1
Gson	8	24	3 / 0	3 / 0	2 / 0	1 / 0
JacksonCore	8	24	7 / 1	6 / 0	2 / 0	0 / 0
JacksonDatabind	8	24	2 / 0	2 / 0	1 / 0	0 / 0
JacksonXml	6	20	2 / 0	2 / 0	1 / 0	0 / 0
Jsoup	8	24	6 / 1	6 / 1	2 / 0	2 / 0
JXPath	8	24	5 / 0	5 / 0	2 / 0	2 / 0

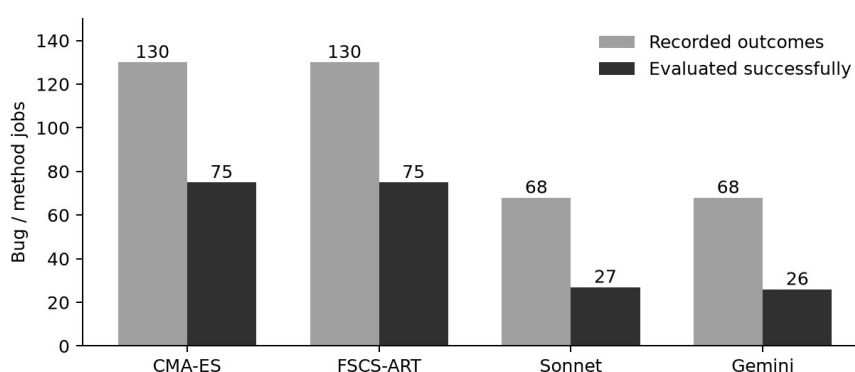
Project	บั๊ก	ผล	CMA	FSCS	Sonnet	Gemini
Lang	7	22	5 / 0	6 / 0	1 / 0	4 / 0
Math	7	22	6 / 0	6 / 1	1 / 1	3 / 1
Mockito	7	22	5 / 1	5 / 1	1 / 1	0 / 0
Time	7	22	5 / 2	5 / 2	2 / 0	3 / 0

ตารางแสดงแต่ละวิธีเป็น จำนวน DONE / จำนวนบั๊กที่พบ fault ตัวเลข 0 / 0 อาจเกิดจากไม่มี suite ที่ประเมินสำเร็จ จึงไม่ตีความว่าอัตราพบ fault เป็น 0% บั๊กที่มีชื่ออยู่ในตารางไม่ได้แปลว่ามีผล AI ครอบคลุมบั๊ก รายการและสถานะเฉพาะบั๊กอยู่ในภาคผนวก ก

เมื่อดูรายโปรเจกต์ Cli และ Closure ไม่มี suite ที่ประเมินสำเร็จเลยในทุกวิธี โดย Cli ทุกบั๊กของอัลกอริทึมเป็น INFRA_ERROR จากปัญหา classpath (ดูหัวข้อ 8.1) ส่วน Closure ส่วนใหญ่เป็น UNSUPPORTED หรือ INVALID_GENERATED_SUITE ของอัลกอริทึม และเป็น INVALID_AFTER_REPAIR ของ AI ในทางตรงข้าม Collections มี DONE สูงสุดของอัลกอริทึม (8 จาก 8 บั๊กในทั้งสองวิธี) ความต่างระหว่าง projects นี้ทำให้ค่ารวมในหัวข้อ 6.2 ขึ้นกับว่าบั๊กใดถูกประเมินได้

6.2 ผลรวมของแต่ละวิธี

วิธี	มีผล	DONE	สำเร็จ	พบ/วัดได้	Fault rate	95% CI (Wilson)
CMA-ES	130	75	57.69%	10/75	13.33%	7.4–22.8%
FSCS-ART	130	75	57.69%	9/75	12.00%	6.4–21.3%
KKU Sonnet 5	68	27	39.71%	3/27	11.11%	3.9–28.1%
KKU Gemini 3.5 Flash Lite	68	26	38.24%	3/26	11.54%	4.0–29.0%



ภาพที่ 2 จำนวนงานที่มีสถานะและจำนวนงานที่ประเมินสำเร็จ

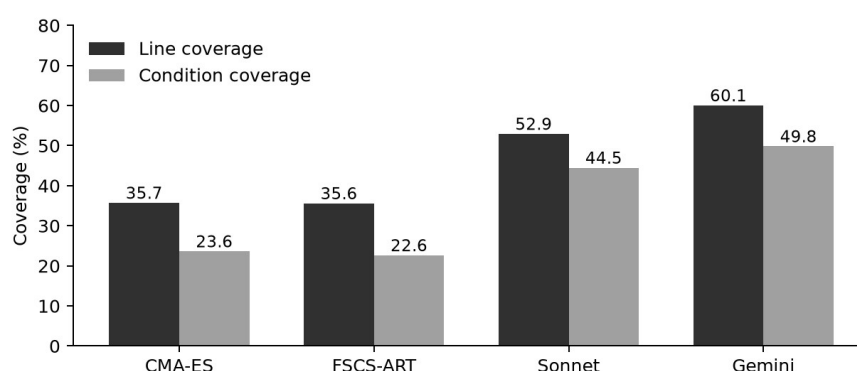
อัลกอริทึมมี outcomes วิธีละ 130 งาน ขณะที่ AI มีวิธีละ 68 งาน จึงไม่ควรเปรียบเทียบจำนวน fault โดยละตัวหาร CMA-ES และ FSCS-ART มีอัตราประเมินสำเร็จเท่ากัน แต่พบ fault ต่างกันหนึ่งบั๊ก ส่วน

Sonnet และ Gemini พบอย่างละสามบักบนชุดที่วัดได้คนละจำนวนและอาจเป็นคนละบัก ช่วงความเชื่อมั่นที่ทับซ้อนกันเกือบทั้งหมดยืนยันว่ายังไม่มีหลักฐานพอจะเลือกผู้ชนะด้าน fault detection

ผลรวมทั้งหมดคือ DONE 203, INVALID_AFTER_REPAIR 61, UNSUPPORTED 84, INFRA_ERROR 31, INVALID_GENERATED_SUITE 6, OUTPUT_INCOMPLETE 2 และ QUOTA_PAUSED 9 รวม 396 outcomes ส่วนอีก 1,604 งานเป็น PENDING งาน AI ที่ไม่ได้ส่ง request ไม่นับเป็นผลล้มเหลวของโมเดล

6.3 ความครอบคลุมของโค้ด

วิธี	Line %	จำนวน	Condition %	จำนวน
CMA-ES	35.74	75	23.63	74
FSCS-ART	35.62	75	22.64	74
KKU Sonnet 5	52.94	27	44.49	27
KKU Gemini 3.5 Flash Lite	60.05	26	49.84	26



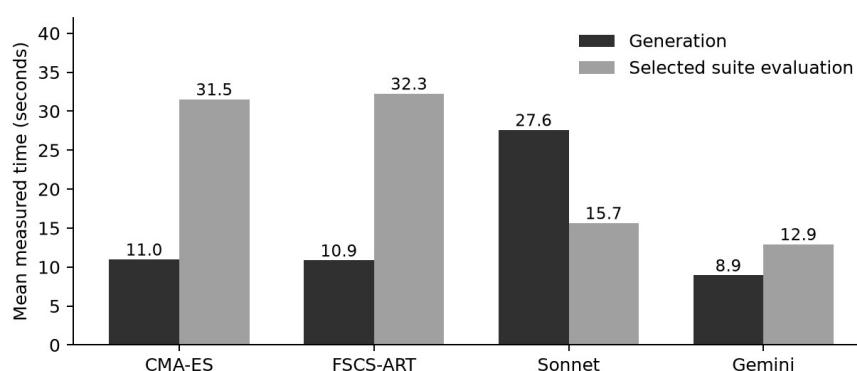
ภาพที่ 3 ค่าเฉลี่ย coverage เฉพาะกรณีที่วัดได้

Gemini มีค่าเฉลี่ย line coverage สูงสุดในภาพรวมที่วัดได้ ตามด้วย Sonnet ส่วน CMA-ES และ FSCS-ART ใกล้กันมาก อย่างไรก็ตาม averages นี้มาจากบักที่วัดได้ 75, 75, 27 และ 26 ตัว จึงมีความต่างของ selection และ target classes ต้องใช้การเปรียบเทียบชุดเดียวกันประกอบก่อนอธิบายความสามารถของวิธี

อัลกอริทึมมี line coverage 75 กรณี แต่ condition coverage 74 กรณี เพราะมีกรณีที่ไม่ได้ใช้ตัวหารของ condition ในค่าเฉลี่ย การรายงานค่าเฉลี่ยต้องระบุจำนวนด้วยเพื่อป้องกันความเข้าใจว่าบักที่ไม่มีค่าได้รับการนับเป็น 0%

6.4 เวลาและทรัพยากร AI

วิธี	สร้าง วินาที	ประเมิน วินาที	จำนวนที่วัด สร้าง/ประเมิน
CMA-ES	10.97	31.53	88 / 88
FSCS-ART	10.85	32.26	88 / 88
KKU Sonnet 5	27.60	15.65	68 / 55
KKU Gemini 3.5 Flash Lite	8.95	12.94	68 / 66



ภาพที่ 4 เวลาเฉลี่ยที่วัดได้ แยกการสร้างและการประเมิน suite

จำนวนที่วัดเวลาในตารางนี้ไม่เท่ากับจำนวน DONE (75, 27 และ 26) เพราะเวลาเฉลี่ยคำนวณจากงานที่มีค่าเวลาบันทึกไว้ในแต่ละขั้นตอน ซึ่งรวมงานที่ไปถึงขั้นประเมินแต่ไม่ได้สถานะ DONE ด้วย ขณะที่ตัววัดอื่นนับเฉพาะ DONE

Sonnet ใช้เวลาสร้างเฉลี่ยสูงกว่าอีกสามวิธีในข้อมูลที่บันทึก ส่วน Gemini ใช้เวลาสร้างเฉลี่ยต่ำกว่า Sonnet แต่ตัวเลขนี้ยังไม่รวม checkout และการรันซ่อม ทุกขั้นตอนเป็นค่าเดียว เวลาประเมินคือ suite ที่เลือกสุดท้าย จำนวน methods และ target ที่ต่างกันส่งผลต่อเวลา จึงไม่ดีความผลนี้เป็นเวลารวมเพื่อทำครบ dataset

AI	Input tokens	Output tokens	คำขอที่บันทึก	Input / คำขอ
KKU Sonnet 5	972,993	250,378	191	≈ 5,094
KKU Gemini 3.5 Flash Lite	761,653	149,950	210	≈ 3,627

AI	Output tokens	คำขอที่บันทึก	Output / คำขอ
KKU Sonnet 5	250,378	191	≈ 1,311
KKU Gemini 3.5 Flash Lite	149,950	210	≈ 714

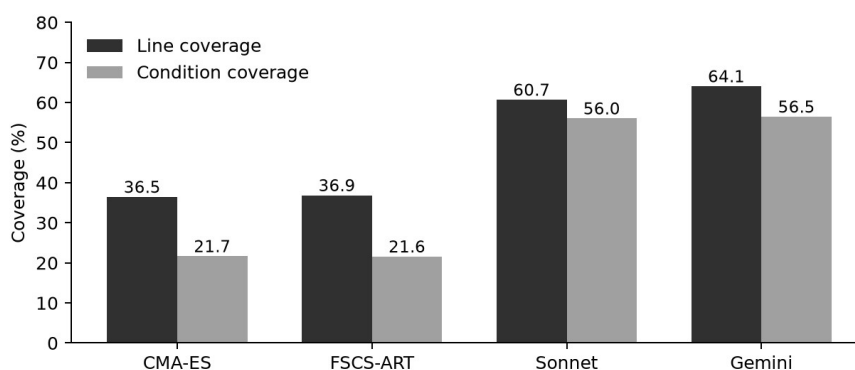
Tokens นับจากค่าที่บริการส่งกลับ บาง Sonnet responses ไม่มี total_tokens จึงรายงาน input/output แยกกัน จำนวนคำขอรวมทั้ง P01-P04 ไม่เท่ากับจำนวนบัก และอาจรวม request attempts ที่ไม่สำเร็จ ค่าเฉลี่ยต่อคำขอจึงเป็นค่าประมาณเพื่อเปรียบเทียบเบื้องต้น โควตา รวมของคีย์ทั้งสิบหรือเวลาที่รีเซตไม่ได้รับการยืนยัน จึงไม่แปลงจำนวนคีย์เป็นจำนวนบัญชีอิสระโดยอัตโนมัติ

7. การวิเคราะห์เปรียบเทียบ

7.1 เปรียบเทียบบนบักชุดเดียวกัน

เลือกเฉพาะบักที่มี DONE ครบทั้งสี่วิธีเพื่อให้คู่เปรียบเทียบใช้บักเดียวกัน ได้ 9 บัก ได้แก่ Chart-4, Collections-1, Csv-1, Csv-2, Gson-3, Jsoup-1, Jsoup-2, JXPath-2, Time-3 การเลือกนี้ทำหลังทดลองเพื่อวิเคราะห์ matched subset ไม่ได้ใช้คัดผลที่ดีที่สุดสำหรับรันเพิ่ม และไม่แทนผลของบักที่ไม่ผ่านในรายงานภาพรวม

วิธี	Line %	Condition %	เทสเฉลี่ย	พบ fault
CMA-ES	36.46	21.67	20.89	0/9
FSCS-ART	36.87	21.59	21.22	0/9
KKU Sonnet 5	60.70	56.04	9.78	0/9
KKU Gemini 3.5 Flash Lite	64.06	56.47	8.33	0/9



ภาพที่ 5 ค่าเฉลี่ย coverage ของ 9 บักที่ประเมินครบสี่วิธี

ในชุดย่อยนี้ Gemini และ Sonnet มี coverage สูงกว่าอัลกอริทึม แม้จำนวนเทสเฉลี่ยต่ำกว่า แต่ทั้งสี่วิธีพบ fault 0 จาก 9 บัก จึงยังไม่มีหลักฐานจาก matched subset นี้ว่า AI ตรวจพบบักได้ดีกว่า ค่า coverage สูงขึ้นแสดงการเข้าถึงบรรทัดและเงื่อนไขที่วัดได้ ไม่ได้ยืนยันว่า assertions จะจับการเปลี่ยนพฤติกรรมจากบัก

ชุดย่อยมีเพียง 9 บั๊ก และรอดจากข้อจำกัดการสร้าง suite ของทุกวิธี ทำให้เกิด survivorship bias ได้ ไม่ทำการทดสอบนัยสำคัญหรืออ้างความเหนือกว่าทั่วไปจากค่าเฉลี่ยนี้

7.2 ความแตกต่างของการตรวจพบบั๊ก

CMA-ES พบ fault ใน Codec-4, Codec-7, Collections-4, Compress-1, Compress-8, JacksonCore-5, Jsoup-8, Mockito-2, Time-1, Time-2

FSCS-ART พบ fault ใน Codec-4, Collections-2, Collections-4, Compress-8, Jsoup-8, Math-6, Mockito-2, Time-1, Time-2

KKU Sonnet 5 พบ fault ใน Collections-4, Math-3, Mockito-2

KKU Gemini 3.5 Flash Lite พบ fault ใน Chart-3, Csv-4, Math-4

รายชื่อแสดงว่าแต่ละวิธีตรวจพบคนละส่วนของบั๊กที่ประเมินได้ เช่น Compress-1 พบด้วย CMA-ES ขณะที่ Collections-2 พบด้วย FSCS-ART และ Chart-3 พบด้วย Gemini การนับผลรวมอย่างเดียวจึงไม่พอ ต้องพิจารณาความถูกต้องของ suite, oracle และการรองรับ target ในแต่ละบั๊กด้วย

7.3 ตัวอย่างหลักฐานที่ตรวจสอบย้อนกลับได้

บั๊ก	วิธี	เทส	Fixed 1 fail	Fixed 2 fail	Buggy fail
Compress-1	CMA-ES	30	0	0	3
Collections-4	Sonnet	13	0	0	1
Csv-4	Gemini	6	0	0	1

Compress-1 ของ CMA-ES รัน 30 methods มี fixed failures เป็น 0 ทั้งสองครั้ง และ buggy failures 3 methods ได้แก่ generated4, generated9 และ generated10 การนับ fault cases จึงเพิ่มหนึ่งบั๊ก ไม่เพิ่มสามบั๊ก line coverage เป็น 104/165 หรือ 63.03% และ condition coverage เป็น 24/59 หรือ 40.68%

Collections-4 ของ Sonnet ใช้ extended suite ที่ประเมินแล้ว โดย line coverage 99/112 หรือ 88.39% และ condition coverage 41/48 หรือ 85.42% ส่วน Csv-4 ของ Gemini พบ fault แม้ line coverage เพียง 4/100 หรือ 4.00% และ condition 0/52 ตัวอย่างหลังแสดงว่าการเข้าถึงพฤติกรรมเฉพาะที่มีบั๊กอาจสำคัญกว่าค่า coverage เฉลี่ยสูงเพียงอย่างเดียว

หลักฐานของแต่ละตัวอย่างอยู่ที่ Experiment/evaluations/<บัก>/<วิธี>/<condition>/run-final/ ซึ่งมี outcome.json, record.json, logs และ suite archive รวมถึง stages ของ base/repair/extension หากมี ไม่ใช่ผล fixed ที่ล้มเหลวของ base เป็นผล buggy ของ suite สุดท้าย

8. ปัญหาและสิ่งที่เรียนรู้

8.1 ปัญหาที่พบจากผลจริง

Sonnet Lang-1 หยุดที่เพดาน output 4,096 tokens ทำให้ Java source ไม่ครบ เก็บสถานะ OUTPUT_INCOMPLETE และคำตอบเต็มไว้ ไม่เรียกซ้ำจนกว่าจะได้ผลที่ผ่าน Sonnet Math-1 ถูกพักก่อน P02 ด้วย operator guard ซึ่งบอกว่าไม่รับคำขอถัดไป ไม่ได้บอกว่า provider เหลือโควตาเป็นศูนย์

Cli-1 พบ NoClassDefFoundError ของ org/hamcrest/SelfDescribing จากปัญหา classpath จึงเก็บเป็น INFRA_ERROR และไม่นับว่าเป็นการตรวจพบบักของ Commons CLI โดยไม่มีวิธีใดได้สถานะ DONE ใน Cli ของ snapshot นี้ ส่วน Closure-1, Closure-3 และ Closure-6 ของ CMA-ES และ FSCS-ART รวม 6 งาน คอมไพล์ไม่ผ่านเพราะ helper ใช้ multi-catch ซึ่ง Java source 6 ไม่รองรับ งานเหล่านี้มีสถานะ INVALID_GENERATED_SUITE และสะท้อนข้อจำกัดของโค้ดที่สร้างกับสภาพแวดล้อมการคอมไพล์ จึงไม่ควรใช้ตัดสินความสามารถในการตรวจพบ fault ของวิธี ส่วน receivers และชนิดข้อมูลที่อยู่นอกขอบเขตของ adapter ต้องเก็บเป็น UNSUPPORTED แม้ขั้นตอน checkout จะสำเร็จ

งาน AI บางชุดมี compile หรือ fixed failures หลังการซ่อมหนึ่งครั้ง จึงเป็น INVALID_AFTER_REPAIR ขณะที่ P04 ที่ใช้ไม่ได้อาจถูกปฏิเสธโดยเก็บ base ที่ valid ไว้ ผลของ suite สุดท้ายต้องอ่าน selected_suite และ selected_measurement คู่กันเพื่อไม่รวมหลายขั้นตอนซ้ำเป็นบักใหม่

เมื่อไม่มีบัญชีผ่านเกณฑ์โควตาพร้อมสำหรับสองโมเดล ตัวรันทำอัลกอริทึมต่อ แต่ไม่เริ่ม AI ใหม่ จึงมี outcomes อัลกอริทึม 130 ต่อวิธีและ AI 68 ต่อวิธี งาน AI ที่ไม่ได้ส่งยังเป็น PENDING ไม่ตีความว่าทั้งสองโมเดลหมดโควตาหรือสร้างเทสไม่สำเร็จทุกงานนั้น

8.2 ผลลัพธ์การเรียนรู้

ได้เรียนรู้ว่าการตรวจ source, การคอมไพล์, การประเมินบน fixed, การตรวจ buggy และการวัด coverage เป็นคนละเงื่อนไข เทสที่ผ่าน fixed ไม่ได้แปลว่าจับบักได้ และค่า coverage สูงไม่รับประกัน fault detection ข้อมูลสถานะและตัวหารจึงมีความสำคัญเท่ากับค่าตัวชี้วัด

การแยก raw evidence ออกจากสรุปผลช่วยแก้ความสับสนระหว่างจำนวนบั๊ก จำนวน test methods จำนวน API calls และจำนวนรอบทดลอง การบันทึก failures ทำให้เห็นต้นทุนของการทำให้ suite ใช้งานได้ และสามารถออกแบบการทดลองรอบถัดไปโดยไม่ลบผลที่ไม่ดีของรอบเดิม

9. ขอบเขต ทรัพยากร และข้อจำกัด

9.1 ขอบเขตที่ดำเนินการแล้ว

ใช้ Java projects จาก Defects4J ทั้ง 17 รายการ มีผลบันทึก 130 บั๊ก โดยไม่อ้างว่าแต่ละวิธีประเมินครบทุกบั๊ก งานที่วางแผนไว้ 500 บั๊กรยังมียังมี PENDING 1,604 คู่บั๊ก/วิธี ไม่ผสมผลเก่าที่ใช้โมเดลหรือ protocol คนละรุ่นเข้ากับ snapshot นี้

9.2 ข้อจำกัดของการเปรียบเทียบ

อัลกอริทึมใช้ fixed-reference oracle แต่ AI ใช้การตีความสัญญาของ API จึงมีข้อมูลและชนิด assertion ต่างกัน จำนวน JUnit methods ไม่เท่ากัน งบ 30 vectors ไม่เทียบตรงกับ token budget ผลค่าเฉลี่ยที่รวม pilot กับ nightly มี condition ต่างกัน และไม่มี generation repeats อิสระเพื่อวัดความแปรปรวน

การตรวจ fixed ผ่านสองครั้งเป็นการตรวจความสามารถในการรันซ้ำของ suite เดิมเบื้องต้น ไม่ยืนยันว่าไม่มี flaky tests ตัวอย่างเช่น suite ของ Sonnet สำหรับ Mockito-2 มีการใช้ Thread.sleep แม้ master prompt ห้ามใช้ sleep แต่ planning context เสนอการรอเวลา จึงมีคำสั่งที่ขัดกัน ส่วน suite ของ Gemini สำหรับ Compress-3 มี assertion ของ byte array ที่ตรวจพฤติกรรมได้จำกัด จึงต้องทบทวนคุณภาพของ oracle เพิ่มเติม การวัด coverage จำกัดที่ modified classes และ counters ที่อ่านได้ ไม่แทน project-wide coverage หรือความถูกต้องของทุก declaration นอกจากนี้ ชุดเปรียบเทียบ 9 บั๊กมีขนาดเล็กและคัดจากบั๊กที่ DONE ครบทั้งสี่วิธี จึงจำกัดการสรุปผลทั่วไป

9.3 ทรัพยากรและสถานที่ดำเนินงาน

การทดลองใช้คอมพิวเตอร์ของผู้ดำเนินงานหนึ่งเครื่องผ่าน WSL และบริการ KUU IntelSphere เก็บไฟล์ใน repository Project_SQA branch Team ไม่มีข้อมูลวัด CPU utilization, RAM peak หรือค่าใช้จ่ายเป็นเงินจริงอย่างครบถ้วน จึงรายงานเฉพาะสเปก เวลา และ tokens ที่วัดได้ ไม่สร้างงบประมาณสมมติเป็นค่าใช้จ่ายจริง

9.4 งานที่ค้างอยู่

ขยายบั๊กที่ยัง PENDING ภายใต้โควตาและเวลาที่ได้รับอนุญาต ตรวจสอบแก้ปัญหา infrastructure ใน condition ใหม่โดยรักษาผลเดิม เพิ่ม generation repeats และ budget configurations เพื่อวัดความแปรปรวน และทบทวน assertions ที่พบ fault ร่วมกับ patch/triggering tests อย่างเป็นระบบ งานเหล่านี้เป็นข้อเสนอการดำเนินการต่อ ไม่ใช่ผลที่ทำแล้วในรายงานนี้

10. สรุปผลและข้อเสนอแนะ

โครงการดำเนินการสร้างและตรวจ unit test suites ด้วย CMA-ES, FSCS-ART, KCU Sonnet 5 และ KCU Gemini 3.5 Flash Lite ได้ผลจริง 396 งานใน 130 บั๊กจาก 17 projects มีข้อมูลครบสัณฐาน 68 บั๊ก และประเมินสำเร็จครบสัณฐาน 9 บั๊ก พร้อมเก็บ source, prompts, configuration, raw results และ hashes สำหรับตรวจสอบย้อนกลับ

ในภาพรวม CMA-ES และ FSCS-ART ประเมินได้วิธีละ 75 บั๊ก และตรวจพบ 10 กับ 9 บั๊ก ส่วน Sonnet และ Gemini ประเมินได้ 27 กับ 26 บั๊ก และตรวจพบวิธีละสามบั๊ก AI มีค่าเฉลี่ย coverage สูงกว่าในชุดที่วัดได้ แต่การเปรียบเทียบชุดบั๊กเดียวกันกับบั๊กไม่พบ fault ของทุกวิธี จึงยังไม่เลือกผู้ชนะด้าน fault detection จากผลนี้

การเลือกเครื่องมือควรพิจารณาว่า target รองรับ input/receiver แบบใด มี oracle ที่เชื่อถือได้หรือไม่ ยอมรับต้นทุนโควตาและเวลาได้เพียงใด และ suite ที่สร้างผ่าน fixed จริงหรือไม่ สำหรับ targets ที่ adapter รองรับ อัลกอริทึมรันซ้ำโดยไม่ใช้โควตา AI ได้ ส่วน AI ช่วยเสนอ assertions เชิงสัญญาได้ แต่ต้องมีการตรวจ compile/fixed และเก็บ raw context ทุกครั้ง

ข้อเสนอรอบถัดไปคือเพิ่มจำนวน matched cases ใช้ขอบเขตและทรัพยากรที่เปรียบเทียบได้ แยก protocol conditions อย่างชัดเจน เพิ่มการสร้างหลาย seed หรือหลายรอบอิสระ และตรวจ classpath ก่อนเปิด batch เพื่อให้เวลาทดลองถูกใช้กับการประเมินจริงมากขึ้น การรายงานผลไม่ผ่านและงานที่ค้างยังเป็นส่วนของผลการทดลองที่จำเป็น

10.1 ความเชื่อมโยงกับสิ่งส่งมอบรอบที่ 2

ข้อกำหนด	หัวข้อ	หลักฐานในชุดส่งมอบ
2.2 (1)	อัลกอริทึม	บทที่ 4–6 และโฟลเดอร์ Algorithm1 / Algorithm2
2.2 (2)	AI และพารามิเตอร์	บทที่ 5–6 และโฟลเดอร์ AI1 / AI2
2.2 (3)	วิเคราะห์และเรียนรู้	บทที่ 7–10 พร้อมผลไม่ผ่าน
2.2 (4)	รายงานและทำซ้ำ	รายงานฉบับนี้ ภาคผนวก และ ZIP หลักฐาน
Presentation และ Demo	PowerPoint 12 slides	demo.py / demo-guide.md

ชุดส่งมอบนี้มีรายงานเนื้อหาครบสำหรับผลที่เก็บได้ แต่ขอบเขตทดลองยังเป็นบางส่วน การมีรายงานครบหัวข้อและหลักฐานไม่เท่ากับการทดลองครบเป้าหมาย 500 บั๊ก ต้องระบุข้อจำกัดนี้ในการนำเสนอด้วย

11. เอกสารอ้างอิง

Chen, T. Y., Leung, H., & Mak, I. K. (2004). Adaptive Random Testing. In Advances in Computer Science – ASIAN 2004: Higher-Level Decision Making (LNCS 3321, pp. 320–329). Springer.
https://doi.org/10.1007/978-3-540-30502-6_23

Hansen, N. (2016). The CMA Evolution Strategy: A Tutorial. arXiv:1604.00772.
<https://arxiv.org/abs/1604.00772>

Just, R., Jalali, D., & Ernst, M. D. (2014). Defects4J: A Database of Existing Faults to Enable Controlled Testing Studies for Java Programs. Proceedings of ISSTA 2014, 437–440.
<https://homes.cs.washington.edu/~mernst/pubs/bug-database-issta2014-abstract.html>

Defects4J maintainers. (2026). Defects4J version 3.0.1 and active bug inventory. สืบค้นวันที่ 4 ตุลาคม 2569 จาก <https://github.com/rjust/defects4j>

มหาวิทยาลัยขอนแก่น. (2569). Project: AI-Assisted Testing vs. Automatic Test Case Generation Algorithms: A Benchmark and Test Coverage Evaluation. เอกสารข้อกำหนดรายวิชา CP353201 Software Quality Assurance ภาคเรียนที่ 1/2569 โดยเฉพาะข้อ 1.5–1.10 และ 2.2.

Project_SQA. (2569). Frozen experiment snapshot วันที่ 4 ตุลาคม 2569 เวลา 21:44 น. branch Team commit 8d02090d9. https://github.com/natchapolp-glitch/Project_SQA/tree/8d02090d9/output/round2-tonight500-20261004-final

หลักฐานหลักที่ใช้คำนวณในรายงาน

- Report/data/final_comparison.csv ตาราง 2,000 งานในขอบเขตที่เลือก
- Report/data/summary.json ผลรวมและตัวหารรายวิธี
- Experiment/snapshot.json และ outcome.json หลักฐานของผลที่ปิดแล้ว
- Experiment/evidence และ Experiment/reproduction/runtime โค้ดและเงื่อนไขที่ตรงไว้
- SHA256SUMS.txt และ Experiment/package-verification.json การตรวจความครบและ hashes

ภาคผนวก ก สถานะรายบัก

D ประเมินสำเร็จ R ไม่ผ่านหลังซ่อม I suite ใช้ไม่ได้ U ไม่รองรับ E infrastructure error
O output ไม่ครบ Q พักโควตา P ยังไม่มีผล เครื่องหมาย * คือพบ fault ในผล DONE

บัก	CMA-ES	FSCS-ART	Sonnet	Gemini
Chart-1	D	D	D	R
Chart-2	U	U	D	D
Chart-3	U	U	D	D*
Chart-4	D	D	D	D
Chart-5	U	U	P	P
Chart-6	D	D	P	P
Chart-7	U	U	P	P
Chart-8	D	D	P	P
Cli-1	E	E	E	E
Cli-2	E	E	E	E
Cli-3	E	E	E	E
Cli-4	E	E	E	E
Cli-5	E	E	P	P
Cli-7	E	E	P	P
Cli-8	E	E	P	P
Cli-9	E	E	P	P
Closure-1	I	I	R	R
Closure-2	U	U	R	R
Closure-3	I	I	R	R
Closure-4	U	U	R	R
Closure-5	U	U	P	P
Closure-6	I	I	P	P
Closure-7	U	U	P	P
Closure-8	U	U	P	P
Codec-1	D	D	R	D
Codec-2	D	D	R	R
Codec-3	D	D	R	R

ชุด	CMA-ES	FSCS-ART	Sonnet	Gemini
Codec-4	D*	D*	R	D
Codec-5	D	D	P	P
Codec-6	U	U	P	P
Codec-7	D*	D	P	P
Codec-8	D	D	P	P
Collections-1	D	D	D	D
Collections-2	D	D*	D	R
Collections-3	D	D	D	R
Collections-4	D*	D*	D*	R
Collections-5	D	D	P	P
Collections-6	D	D	P	P
Collections-7	D	D	P	P
Collections-8	D	D	P	P
Compress-1	D*	D	D	R
Compress-2	D	D	R	R
Compress-3	U	U	D	D
Compress-4	U	U	O	R
Compress-5	D	D	P	P
Compress-6	D	D	P	P
Compress-7	D	D	P	P
Compress-8	D*	D*	P	P
Csv-1	D	D	D	D
Csv-2	D	D	D	D
Csv-3	U	U	R	D
Csv-4	U	U	Q	D*
Csv-5	U	U	P	P
Csv-6	D	D	P	P
Csv-7	U	U	P	P
Csv-8	D	D	P	P
Gson-1	D	D	D	R
Gson-2	U	U	R	R

ชุด	CMA-ES	FSCS-ART	Sonnet	Gemini
Gson-3	D	D	D	D
Gson-4	U	U	Q	R
Gson-5	D	D	P	P
Gson-6	U	U	P	P
Gson-7	U	U	P	P
Gson-8	U	U	P	P
JacksonCore-1	D	D	R	R
JacksonCore-2	D	U	D	R
JacksonCore-3	D	D	D	R
JacksonCore-4	D	D	R	R
JacksonCore-5	D*	D	P	P
JacksonCore-6	D	D	P	P
JacksonCore-7	U	U	P	P
JacksonCore-8	D	D	P	P
JacksonDatabind-1	D	D	R	R
JacksonDatabind-2	U	U	D	R
JacksonDatabind-3	U	U	R	R
JacksonDatabind-4	U	U	R	R
JacksonDatabind-5	U	U	P	P
JacksonDatabind-6	D	D	P	P
JacksonDatabind-7	U	U	P	P
JacksonDatabind-8	U	U	P	P
JacksonXml-1	D	D	Q	R
JacksonXml-2	U	U	D	E
JacksonXml-3	D	D	R	R
JacksonXml-4	U	U	R	R
JacksonXml-5	U	U	P	P
JacksonXml-6	U	U	P	P
Jsoup-1	D	D	D	D
Jsoup-2	D	D	D	D
Jsoup-3	D	D	R	R

ชุด บันทึก	CMA-ES	FSCS-ART	Sonnet	Gemini
Jsoup-4	E	E	R	R
Jsoup-5	D	D	P	P
Jsoup-6	E	E	P	P
Jsoup-7	D	D	P	P
Jsoup-8	D*	D*	P	P
JXPath-1	D	D	R	D
JXPath-2	D	D	D	D
JXPath-3	D	D	D	R
JXPath-4	D	D	Q	E
JXPath-5	D	D	P	P
JXPath-6	U	U	P	P
JXPath-7	U	U	P	P
JXPath-8	U	U	P	P
Lang-1	D	D	O	D
Lang-3	D	D	R	D
Lang-4	U	D	D	D
Lang-5	D	D	Q	D
Lang-6	D	D	P	P
Lang-7	D	D	P	P
Lang-8	U	U	P	P
Math-1	D	D	Q	D
Math-2	D	D	R	D
Math-3	D	D	D*	R
Math-4	U	U	Q	D*
Math-5	D	D	P	P
Math-6	D	D*	P	P
Math-7	D	D	P	P
Mockito-1	D	D	R	R
Mockito-2	D*	D*	D*	R
Mockito-3	D	D	R	R
Mockito-4	D	D	Q	R

บัก	CMA-ES	FSCS-ART	Sonnet	Gemini
Mockito-5	U	U	P	P
Mockito-6	D	D	P	P
Mockito-7	U	U	P	P
Time-1	D*	D*	D	E
Time-2	D*	D*	R	D
Time-3	D	D	D	D
Time-4	D	D	Q	D
Time-5	D	D	P	P
Time-6	U	U	P	P
Time-7	U	U	P	P

ตารางนี้แสดง 130 บักที่มีผลอย่างน้อยหนึ่งวิธี ส่วนบักเป้าหมายที่ยังไม่มีผลเลยอยู่ใน final_comparison.csv การมี P ใน AI ไม่ถูกนับว่าโมเดลล้มเหลว และ D ที่ไม่มี * คือประเมินได้แต่ไม่พบบัก

ภาคผนวก ข การทำซ้ำและหลักฐาน

ข.1 เริ่มจากชุดหลักฐานที่ตรง

ใช้ ZIP หลักฐานเดิม SQA_Round2_500_Target_20261004_FINAL_SNAPSHOT.zip หรือชุดแก้ไข SQA_Round2_CORRECTED_20261005.zip ซึ่งคงผลทดลองเดิมและแก้ไขรายงาน ไม่ควรใช้ Git checkout แทนชุดหลักฐานโดยไม่ตรวจสอบ hash ชุดแก้ไขมี Corrections/verify_evidence.py สำหรับตรวจสอบทั้ง manifest ใหม่และหลักฐานเดิม โดยไม่เรียก AI

SHA-256 ของ ZIP หลักฐานเดิม

SQA_Round2_500_Target_20261004_FINAL_SNAPSHOT.zip:

d27fa596d02e22f96fca0ca8bd6af181404743385a187249d86353b28e31f84b

ข.2 แสดงผลที่บันทึกไว้

จาก root ของ ZIP ใช้ Python แสดงผลของบักโดยไม่เรียก AI เช่น

```
python3 -B Presentation/demo.py --case Csv-1
```

```
python3 -B Presentation/demo.py --case Compress-1 --method cmaes
```

```
python3 -B Presentation/demo.py --case Lang-1 --method kku-claude
```

ข.3 รัน suite เดิมผ่าน Defects4J

ใช้ Java 11 และ Defects4J 3.0.1 บน Ubuntu/WSL เตรียม Csv 1b และ 1f พร้อม compile ทั้งสองรุ่นก่อน แล้วระบุ worktrees และตำแหน่ง Defects4J ให้ตรงเครื่อง output ต้องเป็นไดเรกทอรีใหม่ ไม่เขียนทับหลักฐานเดิม ตัวอย่างคำสั่ง:

```
python3 -B Presentation/demo.py --case Csv-1 --replay \
```

```
--worktrees /tmp/sqa-demo \
```

```
--d4j /absolute/path/to/defects4j \
```

```
--output /tmp/sqa-demo-output-new
```

ใน shell ให้รวมคำสั่งเป็นบรรทัดเดียวหรือใช้ backslash ต่อบรรทัดตาม demo-guide.md การ replay ใช้ archive เดิม ไม่เพิ่มจำนวนบักหรือ generation rounds และไม่ต้องเรียก AI ต้องตรวจสอบ status ของทุกวิธีใน receipt.json เสมอ เพราะ demo เดิมอาจคืน exit code 0 แม้ evaluator ไม่สำเร็จ มีหลักฐานข้อสม Csv-1 ครบทั้งสี่วิธีใน Presentation/rehearsal แต่ไม่ได้บันทึกเป็นวิดีโอการนำเสนอ ส่วนการ replay ใหม่ต้องใช้ output directory ใหม่และไม่เขียนทับหลักฐานเดิม

ข.4 การสร้างใหม่และการตรวจ provenance

การสร้างเทสใหม่ต้องใช้ runtime, seed, budget และ protocol ที่ประกาศ พร้อม output/worktrees ใหม่ ส่วน AI ต้องใช้บัญชี KKU ที่ได้รับอนุญาตและมีโควตา โดยเก็บคีย์ใน environment variable หรือไฟล์ส่วนตัวที่ไม่ commit การมี logs, checksums และ receipt ช่วยตรวจสอบย้อนกลับ แต่ต้องตรวจ oracle และ assertions เพิ่มหากต้องการยืนยันความหมายเชิงวิทยาศาสตร์